

REGISTRO DE ISSUES

Detective's Manual: Caso Blackwood

Sprint 1 e Sprint 2 — Issues #1 a #11

Equipe de Desenvolvimento

Dev 1 — Lucas da Silva Santos

Dev 2 — João David Salomão

Dev 3 — Pollyana de Castro Rodrigues

Disciplinas: Modelagem de Dados / Laboratório de Análise de Algoritmos

Sumário de Issues

#	Título	Responsável	Sprint/Status
#1	Implementar MaxHeap para inventário de pistas	Dev 1 — Lucas	Sprint 1 ✓
#2	Implementar TSP: vizinho mais próximo + 2-opt	Dev 1 — Lucas	Sprint 1 ✓
#3	Implementar Árvore de Decisão com navegação BFS	Dev 1 — Lucas	Sprint 1 ✓
#4	Mecânica de movimento no mapa com névoa de guerra	Dev 3 — Pollyana	Sprint 1 ✓
#5	Coleta de pistas no mapa e inserção no MaxHeap	Dev 3 — Pollyana	Sprint 1 ✓
#6	Sistema de diálogos com escolhas (árvore de decisão)	Dev 3 — Pollyana	Sprint 1 ✓
#7	Tela de acusação usando top 3 do heap	Dev 1 — Lucas	Sprint 1 ✓
#8	Visualização da rota TSP no grid do mapa	Dev 3 — Pollyana	Sprint 2
#9	README técnico + justificativa algorítmica	Dev 2 — João	Sprint 2
#10	Implementar Held-Karp para TSP com $n \leq 15$	Dev 2 — João	Sprint 2
#11	Implementar Models e Seed (MongoDB)	Dev 2 — João	Sprint 1 ✓

SPRINT 1 — Fundação

Meta: estruturas com testes passando + seed rodando + frontend mockado funcionando no browser.

Issue #1 — Implementar MaxHeap para inventário de pistas

Responsável: Dev 1 — Lucas da Silva Santos **Sprint:** Sprint 1 **Labels:** algoritmo, inventário
Milestone: Fundação — Estruturas de Dados

Descrição

Implementar a estrutura MaxHeap usada como inventário de pistas do jogador. Cada pista possui um campo peso (1–10) que determina sua prioridade na investigação. O método top3() retorna as 3 pistas mais relevantes para compor o argumento de acusação final.

Algoritmo

MaxHeap — insert $O(\log n)$, extractMax $O(\log n)$, top3 $O(\log n)$.

Arquivos

- backend/src/structures/MaxHeap.js
- backend/src/structures/MaxHeap.test.js
- backend/src/controllers/partidaController.js (endpoints getInventario, coletarPista, acusar)

Critérios de Aceitação

- ☐ insert(pista) insere corretamente e mantém a propriedade do heap
- ☐ extractMax() remove e retorna sempre o elemento de maior peso
- ☐ top3() retorna as 3 pistas de maior peso na ordem correta sem destruir o heap
- ☐ Heap vazio retorna null sem lançar exceção
- ☐ Testes Jest passando com ao menos 15 pistas inseridas em ordem aleatória
- ☐ Endpoint GET /api/partida/:id/inventario retorna o heap ordenado

Issue #2 — Implementar TSP: vizinho mais próximo + 2-opt

Responsável: Dev 1 — Lucas da Silva Santos **Sprint:** Sprint 1 **Labels:** algoritmo, mapa
Milestone: Fundação — Estruturas de Dados

Descrição

Calcular a rota de investigação que passa por todos os locais com pistas pendentes, partindo da posição atual do detetive, minimizando o custo total de deslocamento. A heurística do vizinho mais próximo constrói a rota inicial em $O(n^2)$ e o algoritmo 2-opt refina essa rota eliminando cruzamentos desnecessários.

Algoritmo

TSP heurístico — Vizinho Mais Próximo $O(n^2)$ + 2-opt $O(n^2)$ por iteração.

Arquivos

- backend/src/structures/TSP.js
- backend/src/structures/TSP.test.js
- backend/src/controllers/partidaController.js (endpoint getRota)

Critérios de Aceitação

- ❑ Algoritmo recebe array de { id, x, y } e retorna array na ordem de visitação
- ❑ Vizinho mais próximo constrói rota inicial em $O(n^2)$
- ❑ 2-opt refina a rota trocando pares de arestas até não haver melhora
- ❑ Rota retornada é mais curta ou igual à ordem aleatória nos casos de teste
- ❑ Funciona com 1 local pendente (retorna lista de 1 elemento)
- ❑ Funciona com array vazio sem lançar exceção
- ❑ Endpoint GET /api/partida/:id/rota retorna a rota calculada

Issue #3 — Implementar Árvore de Decisão com navegação BFS

Responsável: Dev 1 — Lucas da Silva Santos **Sprint:** Sprint 1 **Labels:** algoritmo, gameplay
Milestone: Fundação — Estruturas de Dados

Descrição

Implementar a estrutura de árvore de decisão dos diálogos dos NPCs e o algoritmo BFS para campo de visão do detetive. A árvore representa nós de diálogo (texto do NPC + até 3 escolhas). O BFS calcula as células visíveis a partir da posição atual do detetive, respeitando o raio de 3 células e os limites do grid 10×10.

Algoritmos

Árvore de Decisão — get $O(1)$ via Map / escolher $O(1)$.

BFS — $O(V+E)$ no grid.

Arquivos

- backend/src/structures/ArvoreDecisao.js
- backend/src/structures/BFS.js
- backend/src/structures/ArvoreDecisao.test.js (inclui testes de BFS)
- backend/src/controllers/dialogoController.js
- backend/src/controllers/mapaController.js
- backend/src/routes/partida.js
- backend/src/routes/index.js
- backend/src/server.js

CrITÉrios de AceitaÇão

- ☐ ArvoreDecisao.get(id) retorna o nó correto por ID
- ☐ ArvoreDecisao.escolher(noAtualId, index) navega para o próximo nó corretamente
- ☐ Nó inexistente retorna null sem quebrar a aplicação
- ☐ BFS a partir de (0,0) com raio 3 retorna exatamente as células corretas
- ☐ BFS não inclui células fora do grid 10×10 (sem coordenadas negativas ou ≥ 10)
- ☐ Casos de borda testados: cantos (0,0), (9,9), (0,9), (9,0)
- ☐ Testes Jest passando para árvore e BFS (13 casos)
- ☐ Endpoints /interagir, /escolha e /visao implementados e funcionais

Issue #4 — Mecânica de movimento no mapa com névoa de guerra

Responsável: Dev 3 — Pollyana de Castro Rodrigues **Sprint:** Sprint 1 **Labels:** gameplay, mapa
Milestone: Fundação — Frontend

Descrição

Construir o componente Map com grid 10×10 e mecânica de movimentação do detetive. A névoa de guerra cobre as células além do raio de visão de 3 células calculado pelo BFS do Dev 1. Ao se mover, novas células são reveladas com animação de fade-in.

Arquivos

- frontend/src/app/components/Map/MapScreen.tsx
- frontend/src/app/components/Map/Map.css
- frontend/src/app/components/Map/useMovimento.ts

CrITÉrios de Aceitação

- ☐ Grid 10×10 renderizado com CSS Grid, células de 48×48px
- ☐ Células não reveladas cobertas por overlay escuro (névoa de guerra)
- ☐ Animação de reveal: fade-in de 300ms ao descobrir célula nova
- ☐ Células com pista exibem ícone com animação pulse
- ☐ WASD e teclas de seta movem o detetive
- ☐ Movimento bloqueado ao sair do grid (sem coordenadas negativas ou > 9)
- ☐ Movimento bloqueado enquanto diálogo estiver ativo
- ☐ Integração com endpoint POST /api/partida/:id/mover na Sprint 2

Issue #5 — Coleta de pistas no mapa e inserção no MaxHeap

Responsável: Dev 3 — Pollyana de Castro Rodrigues **Sprint:** Sprint 1 **Labels:** inventário, mapa
Milestone: Fundação — Frontend

Descrição

Ao entrar em célula com pista, o jogador a coleta automaticamente. A pista é inserida no MaxHeap por peso e uma notificação visual desliza da direita com nome e peso da pista. O inventário lateral é atualizado em tempo real.

CrITÉrios de Aceitação

- ☐ Ao entrar em célula com pista, coletar automaticamente ou exibir prompt
- ☐ Chamada ao endpoint POST /api/partida/:id/coletar com o id da pista

- ❑ Card de notificação desliza da direita com nome e peso da pista
- ❑ Pistas com peso ≥ 9 destacadas em vermelho, 7–8 em amarelo, abaixo em verde
- ❑ Célula perde o ícone após coleta
- ❑ Inventário (sidebar) atualiza em tempo real com a nova pista no topo
- ❑ Pistas ordenadas por peso decrescente no inventário (reflexo do MaxHeap)

Issue #6 — Sistema de diálogos com escolhas (árvore de decisão)

Responsável: Dev 3 — Pollyana de Castro Rodrigues **Sprint:** Sprint 1 **Labels:** gameplay, algoritmo

Milestone: Fundação — Frontend

Descrição

Ao entrar em célula com NPC, abrir caixa de diálogo na parte inferior da tela. O texto do NPC aparece com efeito typewriter. Até 3 botões de escolha são exibidos após o texto completar. Cada escolha navega para o próximo nó da Árvore de Decisão e pode revelar pistas ou conceder XP.

Critérios de Aceitação

- ❑ Diálogo abre automaticamente ao entrar em célula com NPC
- ❑ Efeito typewriter: 30ms por caractere, finaliza ao completar
- ❑ Botões de escolha só aparecem após o texto estar completo
- ❑ Ao escolher: fade-out do nó atual, chamada a /escolha, fade-in do próximo nó
- ❑ Se a escolha revelar pista: animação de coleta disparada
- ❑ Nó terminal (sem escolhas) exibe botão "Encerrar conversa"
- ❑ Movimento no mapa bloqueado enquanto diálogo estiver aberto
- ❑ XP ganho por nó visitado reflete no HUD imediatamente

Issue #7 — Tela de acusação usando top 3 do heap

Responsável: Dev 1 — Lucas da Silva Santos (herdada) **Sprint:** Sprint 1 **Labels:** gameplay, inventário

Milestone: Fundação — Frontend

Descrição

Construir a tela final do jogo onde o jogador escolhe um suspeito e o sistema usa as 3 pistas do topo do MaxHeap para montar o argumento. O veredicto indica acerto ou erro e exibe as pistas utilizadas. Issue originalmente atribuída ao Dev 4 (não integrado ao grupo); herdada pelo Dev 1.

Arquivos

- frontend/src/app/components/Accusation/Accusation.tsx
- frontend/src/app/components/Accusation/Accusation.css
- frontend/src/app/components/HUD/HUD.tsx
- frontend/src/app/components/HUD/HUD.css
- frontend/src/app/components/HUD/Inventario.tsx
- frontend/src/app/pages/AccusationPage.tsx

Critérios de Aceitação

- ☐ Tela lista os suspeitos com nome, ícone e motivo resumido
- ☐ Tela de confirmação antes de enviar a acusação: "Tem certeza?"
- ☐ Chamada a POST /api/partida/:id/acusar com suspeitoId
- ☐ Backend retorna as 3 pistas do topo do heap e argumento montado
- ☐ Acerto: tela verde com confetes CSS e argumento exibido
- ☐ Erro: tela vermelha com animação shake e explicação do motivo
- ☐ As 3 pistas usadas pelo heap são exibidas com nome e peso
- ☐ Botão "Jogar novamente" reinicia a partida
- ☐ HUD exibe XP, barra de progresso e level em tempo real
- ☐ Inventário lateral ordena pistas por peso decrescente

Issue #11 — Implementar Models e Seed (MongoDB)

Responsável: Dev 2 — João David Salomão **Sprint:** Sprint 1 **Labels:** banco de dados, documentação

Milestone: Fundação — Modelagem de Dados

Descrição

Criar os schemas Mongoose para as coleções Partida, Pista e Suspeito, e popular o banco MongoDB Atlas com os dados iniciais do caso Blackwood. Esta issue desbloqueia todos os demais desenvolvedores na Sprint 2, pois sem o seed rodando o backend não possui dados para servir.

Arquivos

- backend/src/models/Partida.js
- backend/src/models/Pista.js
- backend/src/models/Suspeito.js
- backend/src/seed/index.js
- backend/src/seed/pistas.js

- `backend/src/seed/suspeitos.js`
- `backend/src/seed/dialogos.js`

Critérios de Aceitação

- ❑ Schema Partida criado com campos: `posicao`, `celulasReveladas`, `pistasColetadas`, `xp`, `status`
 - ❑ Schema Pista criado com campos: `id`, `nome`, `descricao`, `peso`, `celula {x,y}`, `coletada`
 - ❑ Schema Suspeito criado com campos: `id`, `nome`, `relacao`, `motivo`, `segredo`, `ehAssassino`
 - ❑ Seed popula as 10 pistas conforme o manual (posição e peso corretos)
 - ❑ Seed popula os suspeitos com `ehAssassino` correto (Victor Blackwood = true)
 - ❑ Seed popula os nós de diálogo conforme o manual (A0, B0, C0 e ramificações completas)
 - ❑ `node src/seed/index.js` roda sem erros e popula o banco local/Atlas
 - ❑ `GET /api/partida/:id/inventario` retorna dados reais do banco (não mock)
-

SPRINT 2 — Integração e Polimento

Meta: integração completa frontend ↔ backend ↔ banco de dados, Held-Karp implementado, README técnico entregue.

Issue #8 — Visualização da rota TSP no grid do mapa

Responsável: Dev 3 — Pollyana de Castro Rodrigues **Sprint:** Sprint 2 **Labels:** mapa, algoritmo
Milestone: Integração — Frontend

Descrição

Exibir no grid a rota de investigação sugerida pelo TSP como uma linha pontilhada conectando os locais com pistas ainda não coletadas, partindo da posição atual do detetive. A rota é atualizada automaticamente a cada pista coletada.

CrITÉrios de Aceitação

- ☐ Botão "Sugerir Rota" chama GET /api/partida/:id/rota
- ☐ Rota exibida como linha pontilhada CSS sobre o grid, conectando os locais na ordem retornada
- ☐ Linha atualiza ou desaparece ao coletar uma pista do trajeto
- ☐ Locais da rota destacados com borda colorida no grid
- ☐ Se não há pistas pendentes, exibir mensagem "Todas as pistas coletadas"
- ☐ Rota não interfere visualmente com a névoa de guerra

Issue #9 — README técnico + justificativa algorítmica

Responsável: Dev 2 — João David Salomão **Sprint:** Sprint 2 **Labels:** documentação
Milestone: Entrega Final — Documentação

Descrição

Escrever o README técnico do projeto com instruções de instalação, estrutura de pastas, descrição de cada algoritmo implementado e justificativa de escolha com análise de complexidade. Documento exigido como entrega final pelo professor.

CrITÉrios de Aceitação

- ☐ Instruções claras de como rodar o projeto (backend + frontend + seed)
- ☐ Descrição do caso Blackwood e mecânicas do jogo
- ☐ MaxHeap: justificativa + complexidade (insert $O(\log n)$, top3 $O(\log n)$)

- ❑ TSP: justificativa + complexidade ($O(n^2)$ heurística) + comparação com Held-Karp
- ❑ Árvore de Decisão: justificativa + complexidade (get $O(1)$, navegação $O(1)$)
- ❑ BFS: justificativa + complexidade $O(V+E)$ para campo de visão
- ❑ Diagrama de arquitetura (ASCII ou imagem) com fluxo de dados
- ❑ Seção de divisão de responsabilidades por desenvolvedor

Issue #10 — Implementar Held-Karp para TSP com $n \leq 15$

Responsável: Dev 2 — João David Salomão **Sprint:** Sprint 2 **Labels:** algoritmo, mapa
Milestone: Entrega Final — Algoritmos

Descrição

Complementar o TSP heurístico (Issue #2) com a solução exata Held-Karp para casos em que o número de locais com pistas pendentes seja pequeno ($n \leq 15$). A função `calcularRotaTSP` deve selecionar automaticamente o algoritmo conforme o valor de n . No caso Blackwood, o grid possui no máximo 10 pistas — portanto Held-Karp sempre será acionado, garantindo a rota verdadeiramente ótima.

Algoritmo

Held-Karp — $O(n^2 \cdot 2^n)$ para solução exata / $O(n^2)$ heurística para $n > 15$.

Critérios de Aceitação

- ❑ Held-Karp implementado com programação dinâmica em bitmask
- ❑ `calcularRotaTSP` usa Held-Karp quando $n \leq 15$ e heurística quando $n > 15$
- ❑ Para o mesmo input, Held-Karp retorna rota igual ou melhor que vizinho mais próximo + 2-opt
- ❑ Testes Jest comparando resultado exato versus heurístico para $n = 5$ e $n = 10$
- ❑ `TSP.test.js` atualizado com os novos casos de teste
- ❑ Endpoint `GET /api/partida/:id/rota` continua funcionando sem alteração de contrato